สรุปและวิเคราะห์ผล Airflow + Pyspark ETL Benchmark

Created	@March 4, 2026 11:21 AM
Tags	

จากการทดสอบและจับเวลาได้ไฟล์ที่สามารถสรุปเป็นตารางได้ดังนี้:

จากการทดสอบ ETL Pipeline ด้วย Apache Airflow 2.8.0 ร่วมกับ PySpark บน Dataset 3 ขนาด ได้ผลการจับเวลาดังตารางที่ 1 โดยแต่ละแถวแสดงเวลาที่ใช้ในแต่ละขั้นตอน ได้แก่ Extract, Transform และ Load หน่วยเป็นวินาที

dataset	extract_sec	transform_sec	load_sec
10k	0.746	20.3738	32.1863
100k	2.4046	24.1725	108.3891
1m	19.3615	38.9669	955.3723

เพื่อให้เห็นภาพที่ชัดเจนขึ้น ข้อมูลจากตารางได้ถูกนำไปแสดงผลในรูปแบบกราฟ 3 แบบ ดังต่อไปนี้

1. Airflow + PySpark: การเปรียบเทียบเวลาแต่ละ Phase แยกตามขนาดข้อมูล

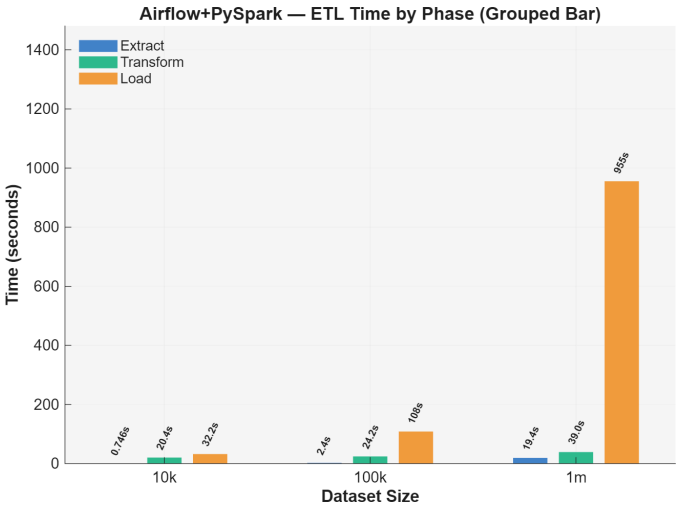

Fig 1 : Airflow + PySpark - การเปรียบเทียบเวลาแต่ละ Phase แยกตามขนาดข้อมูล

จากกราฟ พบว่าขั้นตอน Load ใช้เวลานานที่สุดในทุกขนาดข้อมูล โดยเมื่อข้อมูลมีขนาด 1 ล้านแถว ขั้นตอน Load ใช้เวลาถึง 955 วินาที ในขณะที่ Extract ใช้เพียง 19 วินาที และ Transform ใช้เพียง 39 วินาทีสาเหตุที่ Load ใช้เวลามากที่สุดมาจาก เมื่อ Spark ส่งข้อมูลปริมาณมากไปยังฐานข้อมูลภายนอก ระบบจะใช้ช่องทางเดียวในการส่งข้อมูลทั้งหมด ทำให้ยิ่งข้อมูลมีปริมาณมากเท่าใด กระบวนการนี้ก็ใช้เวลานานมากขึ้นเท่านั้น**[6]**

2. Airflow + PySpark: สัดส่วนของแต่ละ Phase ต่อเวลารวม

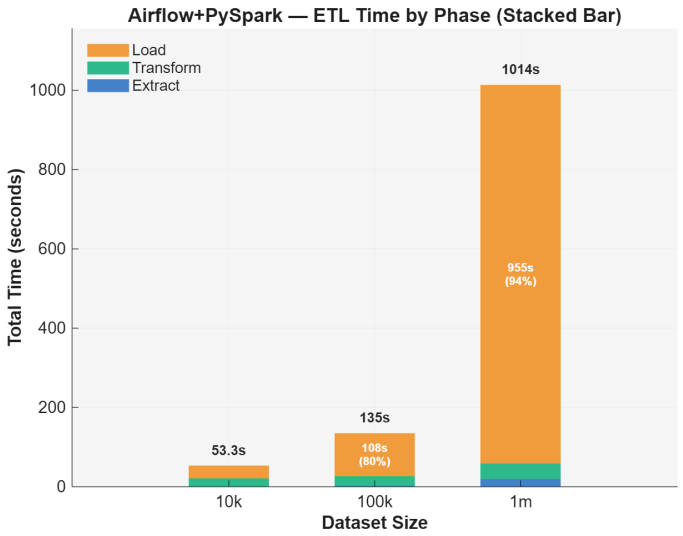

Fig 2 : Airflow + PySpark - สัดส่วนของแต่ละ Phase ต่อเวลารวม

จากกราฟพบว่าขั้นตอน Load ครอบครองเวลารวมมากที่สุดในทุกขนาดข้อมูล และสัดส่วนเพิ่มขึ้นเรื่อยๆ ตามปริมาณข้อมูล โดยที่ข้อมูล 1,000,000 แถว ขั้นตอน Load ใช้เวลาถึง 94% ของเวลารวมทั้งหมด สาเหตุมาจาก PySpark JDBC Writer ในโค้ดไม่ได้ตั้งค่าการกระจายงาน ทำให้ข้อมูลทั้งหมดถูกส่งออกผ่าน JDBC เพียงช่องทางเดียว ซึ่งการส่งข้อมูลทั้งหมดผ่านกระบวนการเดียวจะใช้เวลานานมากเมื่อข้อมูลมีปริมาณมาก และทรัพยากรที่เหลืออยู่ในระบบก็ไม่ได้ถูกนำมาใช้งาน ยิ่งข้อมูลมีปริมาณมากขึ้นเท่าใด เวลาของขั้นตอน Load ก็เพิ่มขึ้นในอัตราที่สูงกว่าขั้นตอนอื่นอย่างชัดเจน

3. Airflow + PySpark: อัตราการขยายตัวของแต่ละ Phase (Scalability)

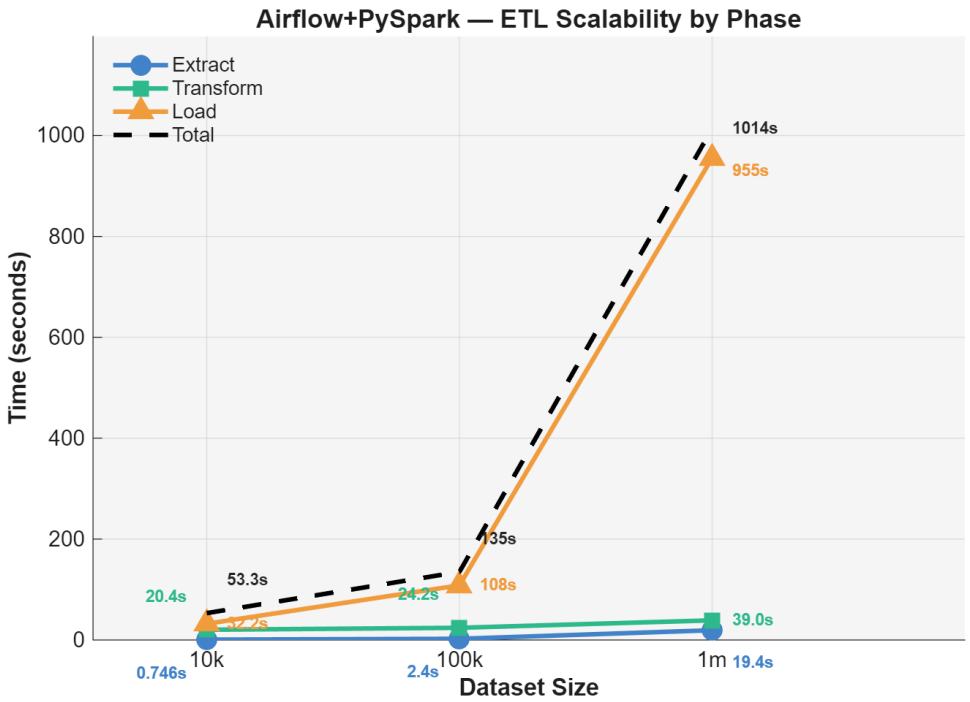

Fig 3 : Airflow + PySpark - อัตราการขยายตัวของแต่ละ Phase

จากกราฟพบว่า **ขั้นตอน Load มีเส้นชันที่สุด** โดยเพิ่มขึ้นประมาณ 30 เท่า (955 ÷ 32.2 = 29.6 เท่า ≈ 30 เท่า) เมื่อข้อมูลเพิ่มขึ้น 100 เท่า สาเหตุที่ Load เพิ่มขึ้นเร็วที่สุดนั้น เมื่อเขียนข้อมูลไปยังฐานข้อมูลผ่าน JDBC นั้น PySpark ใช้จำนวน Partition ที่มีอยู่ในหน่วยความจำเพื่อควบคุมการทำงานพร้อมกัน**[7]** เมื่อไม่ได้ตั้งค่าการกระจายงาน ข้อมูลทั้งหมดจึงถูกส่งออกผ่านช่องทางเดียว

References

[6]S. Asad, “Spark — Parallelization of Reading Data from JDBC sources,” *Medium*, 16-Apr-2023. [Online]. Available: https://medium.com/@suffyan.asad1/spark-parallelization-of-reading-data-from-jdbc-sources-e8b35e94cb82.

[7]*Databricks.com*. [Online]. Available: https://community.databricks.com/t5/data-engineering/speed-up-jdbc-write-from-databricks-notebook-to-ms-sql-server/td-p/45114.